JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Robotics

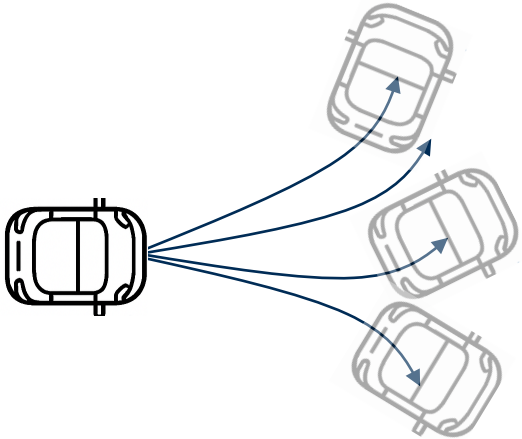
State Estimation

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

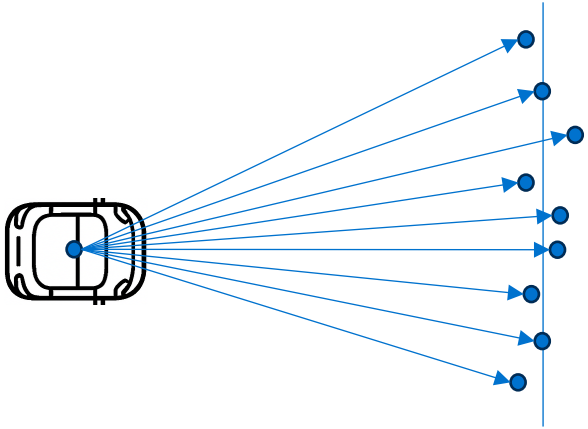
Introduction to Robotics

Probability Refresher

Why a probabilistic approach for localization



Uncertainty in action outcome

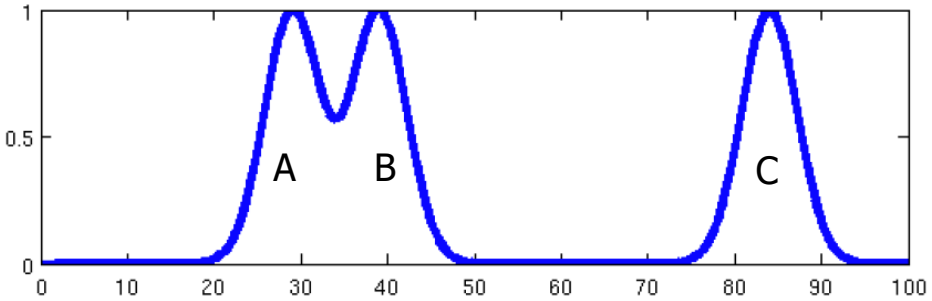


Noise in sensor measurements

Examples of probability distributions (1)

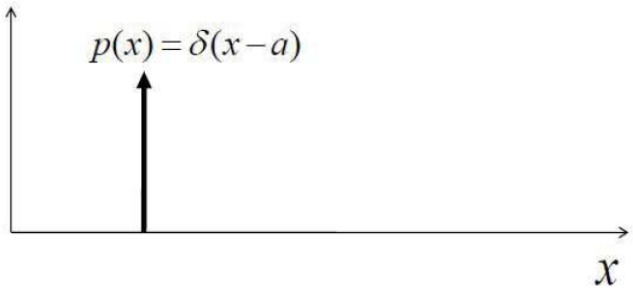
- Example 1: Uniform distribution – the robot does not know where it is: the information about the robot configuration is null:

- Example 2 multi-modal distribution – the robot is in the region around A, B, or C:



Examples of probability distributions (2)

Example 3: Dirac Distribution – the robot has a probability of 1.0 (i.e 100%) to be at a given position

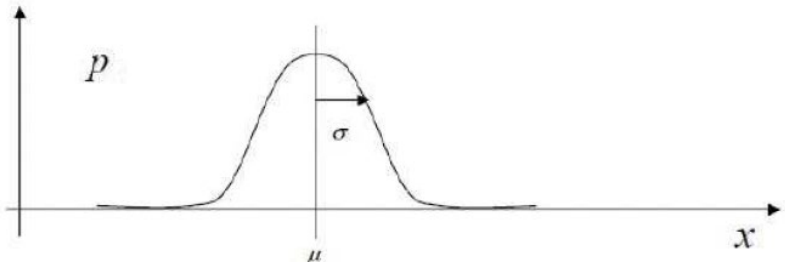


$$\delta(x) = \begin{cases} \infty. \text{ if } x = 0 \\ 0 \text{ otherwise} \end{cases}$$

With $\int_{-\infty}^{+\infty} \delta(x)dx = 1$

Examples of probability distributions (3)

- Example 4: Gaussian distribution – also called normal distribution N(μ,σ):



$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where μ is the mean value and σ is the standard deviation

Axioms of Probability Theory

$P(A)$ denotes the probability that proposition A is true

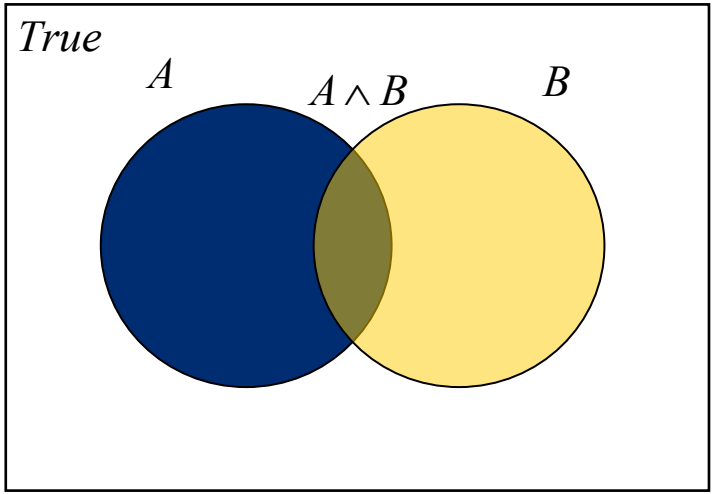
1. $0 \leq P(A) \leq 1$

2. $P(True) = 1$ $\qquad$ $P(False) = 0$

3. $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$

A Closer Look at Axiom 3

$$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$$



Joint and Conditional Probability

- $P(X = x \; and \; Y = y) \; = \; P(x, y)$

- If X and Y are independent then

$$P(x, y) \; = \; P(x) \, P(y)$$

- $P(x \mid y)$ is the probability of *x* given *y*

$$P(x \mid y) \; = \; P(x, y) \; / \; P(y)$$

$$P(x, y) \; = \; P(x \mid y) \, P(y)$$

- If X and Y are independent then

$$P(x \mid y) \; = \; P(x)$$

Law of Total Probability, Marginals

Discrete case

$$\sum_x P(x) = 1$$

$$P(x) = \sum_y P(x, y)$$

$$P(x) = \sum_y P(x \mid y) P(y)$$

Continuous case

$$\int p(x)\, dx = 1$$

$$p(x) = \int p(x, y)\, dy$$

$$p(x) = \int p(x \mid y) p(y)\, dy$$

Bayes Formula

$$P(x, y) = P(x \mid y)P(y) = P(y \mid x)P(x)$$

$$\Rightarrow$$

$$\boxed{P(x \mid y) = \frac{P(y \mid x)\ P(x)}{P(y)} = \frac{\text{likelihood} \cdot \text{prior}}{\text{evidence}}}$$

Normalization

$$P(x \mid y) = \frac{P(y \mid x)\ P(x)}{P(y)} = \eta\ P(y \mid x)\, P(x)$$

$$\eta = P(y)^{-1} = \frac{1}{\sum_x P(y \mid x) P(x)}$$

Algorithm:

$$\forall x : \mathrm{aux}_{x|y} = P(y \mid x)\ P(x)$$

$$\eta = \frac{1}{\sum_x \mathrm{aux}_{x|y}}$$

$$\forall x : P(x \mid y) = \eta\ \mathrm{aux}_{x|y}$$

Conditioning

- Law of total probability:

$$P(x) = \int P(x, z) dz$$

$$P(x) = \int P(x \mid z) P(z) dz$$

$$P(x \mid y) = \int P(x \mid y, z) P(z \mid y) \, dz$$

Bayes Rule with Background Knowledge

$$P(x \mid y, z) = \frac{P(y \mid x, z)\ P(x \mid z)}{P(y \mid z)}$$

Conditional Independence

$$P(x, y \mid z) = P(x \mid z) P(y \mid z)$$

equivalent to

$$P(x \mid z) = P(x \mid z, y)$$

and

$$P(y \mid z) = P(y \mid z, x)$$

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Robotics

Bayes Filter

Components of our system

State (x_t) – Collection of variables sufficient to describe the future

Action (u_t) – Inputs which affect how the state changes from one time to another

Measurement (z_t) – Sensor values that provide information about the current state

	Deterministic form	Probabilistic form
Transition function	$x_{t+1} = f(x_t, u_{t+1})$	$P(x_t \mid x_{t-1}, u_t)$
Measurement function	$z_t = h(x_t)$	$P(z_t \mid x_t)$

Fundamental Problem: State is hidden

All the robot sees is a the history of actions and measurements

$$u_1, z_1, u_2, z_2, \ldots\ldots$$

Challenge: State x_t is hidden.

Solution: Estimate belief over state

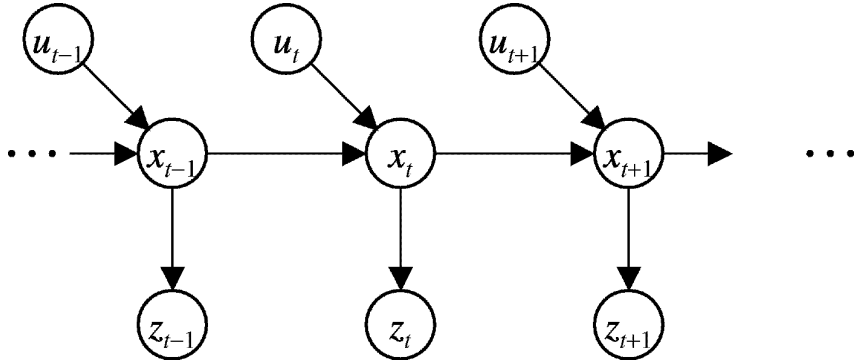
Belief is a probability of each possible state given history

$$bel(x_t) = P(x_t | z_{1:t}, u_{1:t})$$

Also called the Posterior / Information State / State of Knowledge

Markov Property

A Markovian system is one where the next state x_{t+1} can be determined entirely by the previous state x_t and action a_t. History does not matter!



$$p(x_t \mid x_{1:t-1}, z_{1:t-1}, u_{1:t}) = p(x_t \mid x_{t-1}, u_t)$$

$$p(z_t \mid x_{0:t}, z_{1:t}, u_{1:t}) = p(z_t \mid x_t)$$

Underlying Assumptions

- Static world
- Independent noise
- Perfect model, no approximation errors

Integrating the Outcome of Actions

Continuous case:

$$P(x \mid u) = \int P(x \mid u, x')P(x')dx'$$

Discrete case:

$$P(x \mid u) = \sum P(x \mid u, x')P(x')$$

Causal vs. Diagnostic

- $P(x_t|z_t)$ is diagnostic
- $P(z_t|x_t)$ is causal
- Often causal knowledge is easier to obtain.
- Bayes rule allows us to use causal knowledge.

$$P(x \mid z) = \frac{P(z \mid x)P(x)}{P(z)}$$

Derivation of the Bayes Filters

$$\boxed{bel(x_t)} = P(x_t \mid u_1, z_1 \ldots, u_t, z_t)$$

Bayes $$= \eta \; P(z_t \mid x_t, u_1, z_1, \ldots, u_t) \; P(x_t \mid u_1, z_1, \ldots, u_t)$$

Markov $$= \eta \; P(z_t \mid x_t) \; P(x_t \mid u_1, z_1, \ldots, u_t)$$

Total prob. $$= \eta \; P(z_t \mid x_t) \int P(x_t \mid u_1, z_1, \ldots, u_t, x_{t-1}) P(x_{t-1} \mid u_1, z_1, \ldots, u_t) \, dx_{t-1}$$

Markov $$= \eta \; P(z_t \mid x_t) \int P(x_t \mid u_t, x_{t-1}) \; P(x_{t-1} \mid u_1, z_1, \ldots, u_t) \, dx_{t-1}$$

$$\boxed{= \eta \; P(z_t \mid x_t) \int P(x_t \mid u_t, x_{t-1}) \; Bel(x_{t-1}) \, dx_{t-1}}$$

Basic Steps of a Bayes Filter

Step 0: Start with the belief at time step t-1

$$bel(x_{t-1})$$

Step 1: Prediction – Apply the transition model to modify the belief

$$\overline{bel}(x_t) = \int P(x_t, |u_t, x_{t-1}) bel(x_{t-1}) dx_{t-1}$$ Prior

Step 2: Correction – apply the Bayes rule given the latest measurement

$$bel(x_t) = \eta P(z_t | x_t) \overline{bel}(x_t)$$ Posterior

Example - Robot Opening Door Example

Goal: Try to determine the state of the door given noisy actions and sensing

- State Space

$$X = \{Open, Closed\}$$

- Action Space

$$A = \{Push, Wait\}$$

Sensor Model

Assume the sensor is very noisy

$$P(z_t = open \mid x_t = open) = 0.6$$

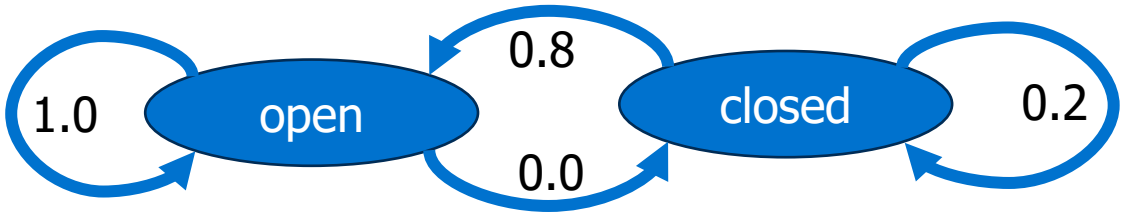
$$P(z_t = closed \mid x_t = open) = 0.4$$

$$P(z_t = open \mid x_t = closed) = 0.2$$

$$P(z_t = closed \mid x_t = closed) = 0.8$$

Transition Model

Assume the robot is clumsy (80% success rate)



State transition diagram for Push

$$P(x_t = open \mid u_t = push, x_{t-1} = closed) = 0.8$$

$$P(x_t = closed \mid u_t = push, x_{t-1} = closed) = 0.2$$

Waiting maintains the same state 100% of the time

Prediction Step 0

Given the initial belief states

$$bel(x_0 = open) = 0.5$$

$$bel(x_0 = closed) = 0.5$$

Allow the robot to take the Wait action.
Our new belief is as follows

$$\overline{bel}(x_1) = \sum p(x_1|u_1, x_0) bel(x_0)$$

$$= p(x_1|u_1 = wait, x_0 = open) bel(x_0 = open) +$$

$$p(x_1|u_1 = wait, x_0 = closed) bel(x_0 = closed)$$

Prediction Step 1

$o = open$
$s = push$
$w = wait$
$c = closed$

$$\overline{bel}(x_1 = o) = p(x_1 = o | u_1 = w, x_0 = o) bel(x_0 = o) + p(x_1 = o | u_1 = w, x_0 = c) bel(x_0 = c)$$

$$\overline{bel}(x_1 = o) = 1 \cdot 0.5 + 0 \cdot 0.5 = 0.5$$

$$\overline{bel}(x_1 = c) = p(x_1 = c | u_1 = w, x_0 = o) bel(x_0 = o) + p(x_1 = c | u_1 = w, x_0 = c) bel(x_0 = c)$$

$$\overline{bel}(x_1 = o) = 1 \cdot 0.5 + 0 \cdot 0.5 = 0.5$$

Correction Step 1

$o = open$
$s = push$
$w = wait$
$c = closed$

Now the robot takes the following measurement

$$z_1 = open$$

We apply the correction to our belief

$$bel(x_1) = \eta p(z_1|x_1)\overline{bel}(x_1)$$

$$bel(x_1 = o) = \eta p(z_1 = o|x_1 = o)\overline{bel}(x_1 = o)$$
$$= \eta \cdot 0.6 \cdot 0.5 = \eta \cdot 0.3$$

$$bel(x_1 = c) = \eta p(z_1 = o|x_1 = c)\overline{bel}(x_1 = c)$$
$$= \eta \cdot 0.2 \cdot 0.5 = \eta \cdot 0.1$$

$$\eta = \frac{1}{0.1 + 0.3} = 2.5$$

$o = open$
$s = push$
$w = wait$
$c = closed$

Final Belief

This provides the final belief states for the system

$$bel(x_1 = o) = \eta p(z_1 = o | x_1 = o)\overline{bel}(x_1 = o)$$
$$= \eta \cdot 0.3 = 0.75$$
$$bel(x_1 = c) = \eta p(z_1 = o | x_1 = c)\overline{bel}(x_1 = c)$$
$$= \eta \cdot 0.1 = 0.25$$

Challenge: compute the probabilities after $u_2 = push$ and $z_2 = open$

Asynchronous Bayes Filter Algorithm

Input: Datapoint d, Current belief $bel(x)$

Output: Updated belief $bel'(x)$

Process:

If d is measurement z **then**

for all x

$$bel'(x) = P(z|x)bel(x)$$

$$bel'(x) = \mathrm{Normalize}(\mathrm{bel}'(\mathrm{x}))$$

Else if d is control u **then**

for all x

$$bel'(x) = \sum_{x_{old}} P(x|x_{old}, u)bel(x_{old})$$

Return $bel'(\mathrm{x})$

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Robotics

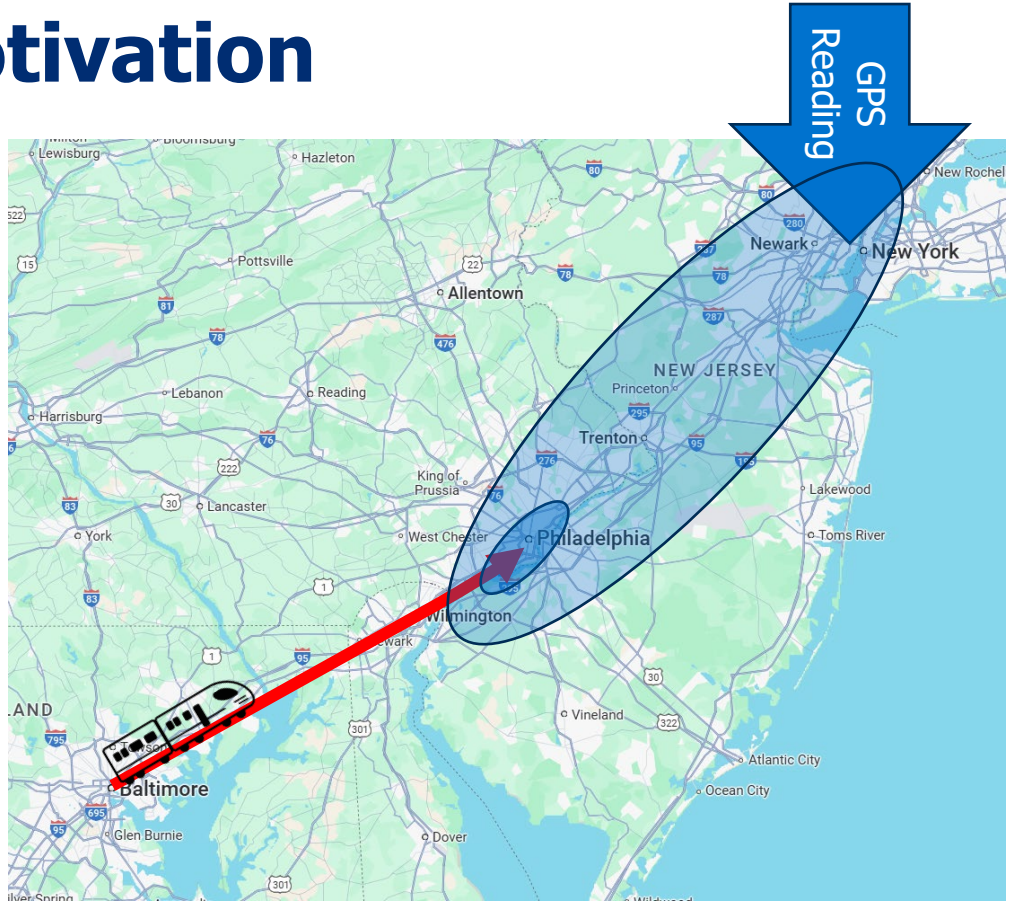
Kalman Filter

Kalman Filter - Motivation

- Bayes Filter - Discrete spaces, Multiple Hypothesis
- Kalman Filter – Continuous spaces, Single Hypothesis
- Very Fast – Real-time performance
- Supports stochastic motion and sensing
- Provides an analytical solution for handling discrepancies between sensors and predictions
- Can handle scaling to high dimension spaces

Kalman Filter - Motivation

- Imagine you taking the train to Philadelphia from JHU.
- You fall asleep and when you wake up you check your GPS
- The GPS says you are in New York.
- Your watch says only an hour has passed
- Which do you trust?
- What if you tried to estimate time from the position of the sun?



Linear Systems

Model the robot as a discrete-time linear dynamical system with the following process and sensor model

$$x_{k+1} = Ax_k + Bu_k + v_k$$
$$z_k = Hx_k + w_k$$

Where,

x_k is the current state $\left(x_k = \text{state at timestep } k\right)$

u_k is the current control input

v_k is (Gaussian) process noise

z_k is the sensor measurement

w_k is (Gaussian) sensor noise

Simple Observer

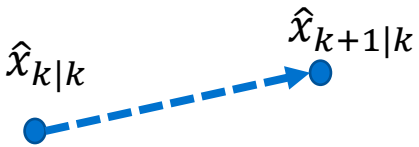
First, let's solve for a system without any noise

Given a linear dynamical system

$$x_{k+1} = Ax_k + Bu_k$$
$$y_k = Hx_k$$

We can insert our current estimate $\hat{x}_{k|k}$ to make the following prediction.

$$\hat{x}_{k+1|k} = A\hat{x}_{k|k} + Bu_k$$



Simple Observer (2)

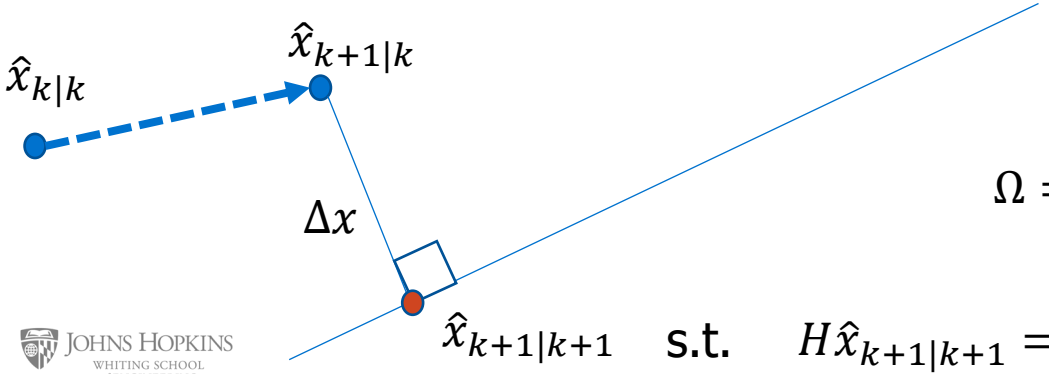
There are many possible values of x which could give thee sensor measurement z_{k+1}

These lie upon the following manifold

$$\Omega = \left\{ x \in \mathbb{R}^n \;\middle|\; Hx_k = z_{k+1} \right.$$

Our goal, is to find the smallest change to our prediction $\hat{x}_{k+1|k}$ that will allow $\hat{x}_{k+1|k+1} \in \Omega$

This value is given by Δx and is perpendicular to the manifold Ω



$$\Omega = \{x \in \mathbb{R}^n | Hx_k = z_{k+1}$$

s.t. $\quad H\hat{x}_{k+1|k+1} = z_{k+1}$

Solving for Δx

We need to find a value of Δx which is orthogonal to Ω s.t. $a^T \Delta x = 0$ for any a parallel to Ω

We leverage the following fact from linear algebra.

1. A vector $a \in \mathbb{R}^n$ is parallel to Ω if and only if $Ha = 0$ The set of all such a is called the **null space** of H and is denoted by $null(H)$
2. A vector $b \in \mathbb{R}^n$ is orthogonal to every vector in the space $null(H)$ if and only if b is in the **column space** of H^T where the column space of H^T is denoted by $column(H^T)$ and is defined to be the span of the columns of H^T

Therefore $b = H^T \gamma$ for some $\gamma \in \mathbb{R}^p$

Giving us

$$\Delta x = H^T \gamma$$

Solving for Δx (2)

Given $\Delta x = H^T \gamma$ now we must solve for γ

First we define the innovation error v to be the difference between the actual output z_{k+1} and the predicted output $H\hat{x}_{k+1|k}$

$$v = z_{k+1} - H\hat{x}_{k+1|k}$$

We assume that there is linear relationship between this error and γ s.t.

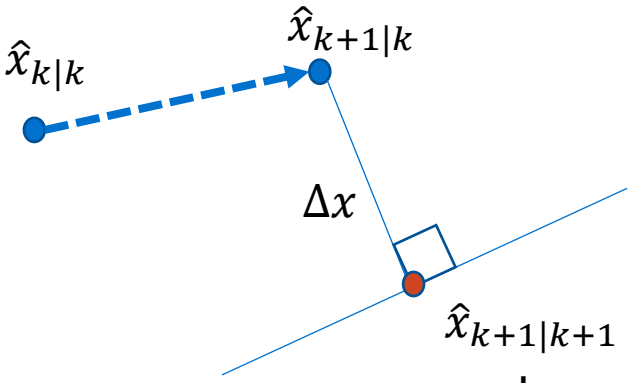
$$\gamma = Kv$$

Which gives us

$$\Delta x = H^T K v$$

$$\Delta x = H^T K \left(z_{k+1} - H\hat{x}_{k+1|k} \right)$$

Solving for Δx (3)



s.t.

$$H\hat{x}_{k+1|k+1} = z_{k+1}$$

Given $\Delta x = H^T K\left(z_{k+1} - H\hat{x}_{k+1|k}\right)$ we must now solve for K

We want to find a K s.t.

$$H\left(\hat{x}_{k+1|k} + \Delta x\right) = z_{k+1}$$

Substituting in our equation for $\Delta x = H^T K v$ gives us

$$H\left(\hat{x}_{k+1|k} + H^T K v\right) = z_{k_+1}$$

$$H\left(H^T K v\right) = z_{k+1} - H\hat{x}_{k+1|k} = v$$

Therefore

$$K = \left(HH^T\right)^{-1}$$

Matrix inverse

Two-step Process for the Simple Model

Prediction

$$\hat{x}_{k+1|k} = A_k \hat{x}_{k|k} + B_k u_k$$

Update

$$\hat{x}_{k+1|k+1} = \hat{x}_{k+1|k} + H^T \left(HH^T \right)^{-1} \left(z_{k+1} - H\hat{x}_{k+1|k} \right)$$

Issues

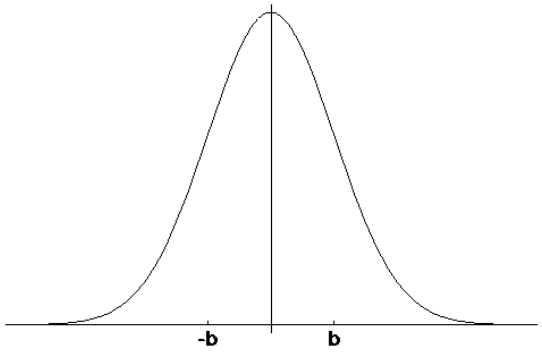
- Errors parallel to Ω are never corrected
- Real sensors have noise (measurement noise)
- Real dynamics have noise (random forces, wheel slip, etc)

Gaussian Distribution

Univariate

$$p(x) \sim N\left(\mu, \sigma^2\right)$$

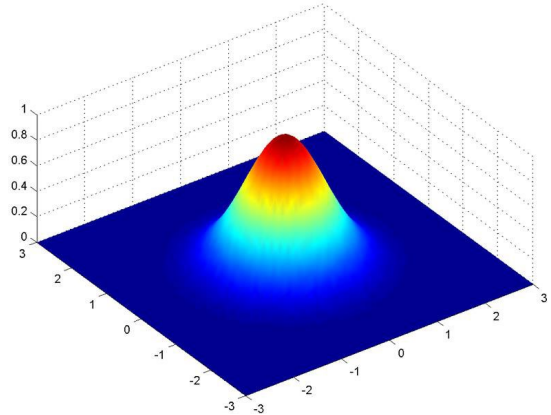
$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{\frac{-(x-\mu)^2}{2\sigma^2}}$$



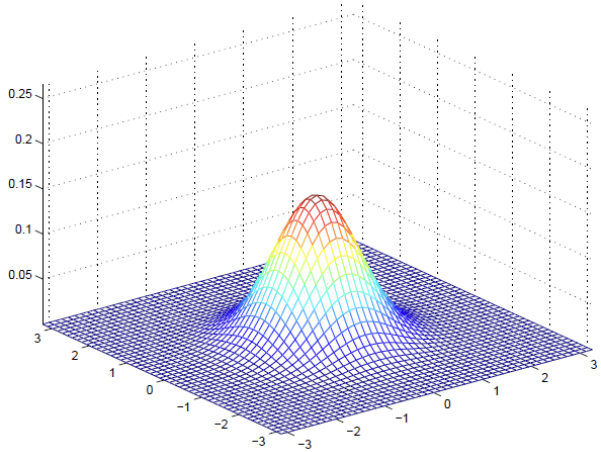
Multivariate

$$p(\boldsymbol{x}) \sim N(\boldsymbol{\mu}, \Sigma)$$

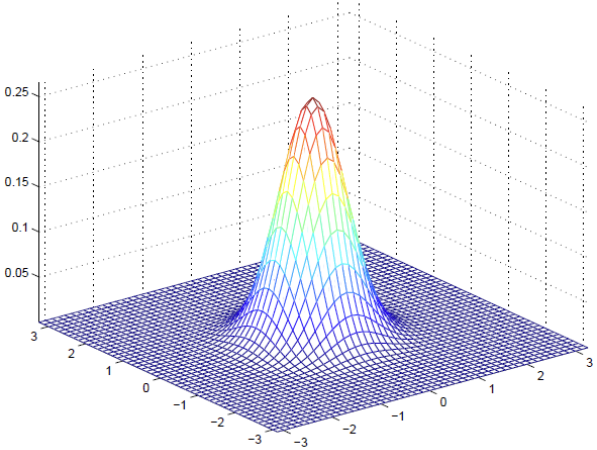
$$p(\boldsymbol{x}) = \frac{1}{(2\pi)^{d/2} \, |\Sigma|^{1/2}} e^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \Sigma^{-1} (\boldsymbol{x}-\boldsymbol{\mu})}$$



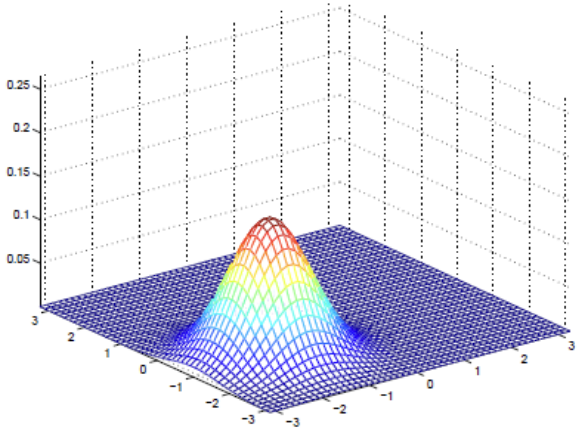
Multivariate Gaussians: Examples



$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

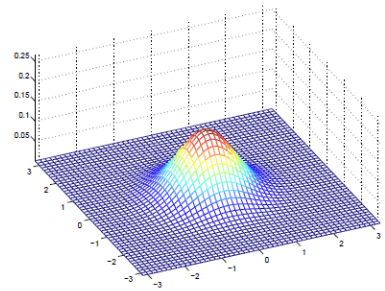


$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Sigma = \begin{bmatrix} 0.6 & 0 \\ 0 & 0.6 \end{bmatrix}$$

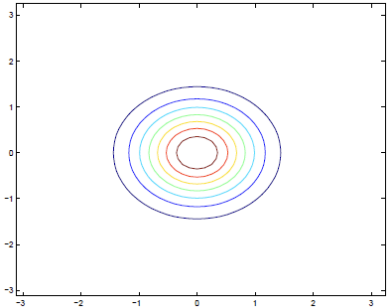


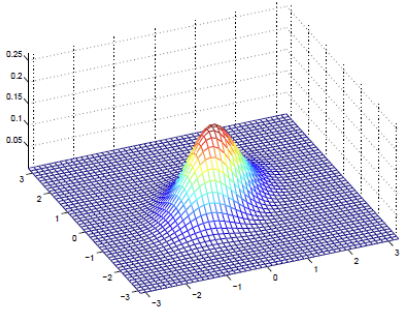
$$\mu = \begin{bmatrix} -1 \\ -1.5 \end{bmatrix} \Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Multivariate Gaussians: Examples (2)

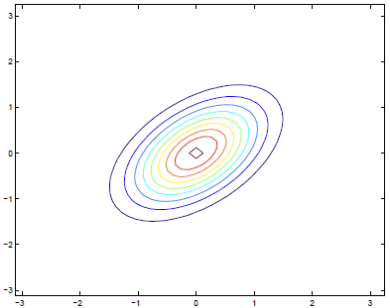


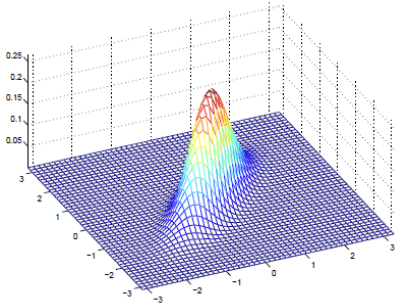
$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Sigma = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$



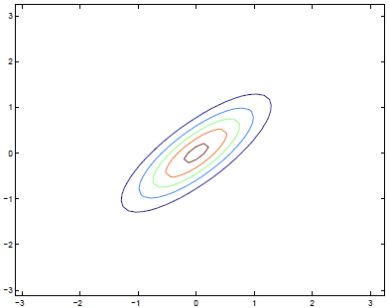


$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$$





$$\mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \Sigma = \begin{bmatrix} 1 & 0.8 \\ 0.8 & 1 \end{bmatrix}$$



Properties of Gaussians

A linear transformation applied to a Gaussian results in a Gaussian!

$$\left.\begin{aligned} X &\sim N\left(\mu,\sigma^2\right) \\ Y &= aX+b \end{aligned}\right\} \rightarrow Y \sim N\left(a\mu+b, a^2\sigma^2\right)$$

$$\left.\begin{aligned} X_1 &\sim N\left(\mu_1,\sigma_1^2\right) \\ X_2 &\sim N\left(\mu_2,\sigma_2^2\right) \end{aligned}\right\} \rightarrow X_1+X_2 \sim N\left(\mu_1+\mu_2, \sigma_1^2+\sigma_2^2\right)$$

$$\left.\begin{aligned} X_1 &\sim N\left(\mu_1,\sigma_1^2\right) \\ X_2 &\sim N\left(\mu_2,\sigma_2^2\right) \end{aligned}\right\} \rightarrow X_1X_2 \sim N\left(\mu_1+Q\left(\mu_2-\mu_1\right), \sigma_1-Q\sigma_1\right)$$

$$Q=\sigma_1\left(\sigma_1+\sigma_2\right)^{-1}$$

The same holds for multivariate Gaussians

Process Noise Model

We now treat our estimate as a multi-variate gaussian.

We now have 2 values to track; a mean state $\overline{x}_k$ and a prior covariance Σ_k

Define prior error between true state and prior estimate

$$e_k^{'} = x_k - \widehat{x_k^{'}}$$

And prior covariance as

$$\Sigma_k = \mathrm{E}\left(e_k^{'} e_k^{'T}\right)$$

Updating our process model to include process noise v_t gives us

$$x_{k+1} = Ax_k + Bu_k + \mathrm{v}_k$$
$$z_k = Hx_k$$

Updating the Covariance

For Σ_t

In addition to updating our estimate $\hat{x}_{k+1}$ we must update the covariance of our estimate Σ_{k+1}

We will simplify our notation here to refer to $\hat{x}_{k+1|k} = \hat{x}'$ and $\hat{x}_{k|k} = \hat{x}_k$

Since $E(\mathrm{v}_k) = 0$ (v_k is Gaussian noise)

$$\hat{x}'_{k+1} = A\hat{x}_k + Bu_k$$

And

$$\Sigma'_{k+1} = \mathrm{E}\left[\left(x_{k+1} - \hat{x}'_{k+1}\right)\left(x_{k+1} - \hat{x}'_{k+1}\right)^T\right]$$

$$\Sigma'_{k+1} = \mathrm{E}\left[A\left(x_k - \hat{x}'_k\right)\left(x_k - \hat{x}'_k\right)^T A^T + 2A\left(x_k - \hat{x}'_k\right)\mathrm{v}_k + \mathrm{v}_k\mathrm{v}_k^T\right]$$

Since v_t is independent of x_t

$$\mathrm{E}\left[\left(x_k - \hat{x}'_k\right)\mathrm{v}_k\right] = \mathrm{E}\left[\left(x_k - \hat{x}'_k\right)\right]\mathrm{E}\left[\mathrm{v}_k\right] = 0$$

$$\Sigma'_{k+1} = A^T\mathrm{E}\left[\left(x_k - \hat{x}'_k\right)\left(x_k - \hat{x}'_k\right)^T\right]A^T + \mathrm{E}\left[\mathrm{v}_k\mathrm{v}_k^T\right]$$

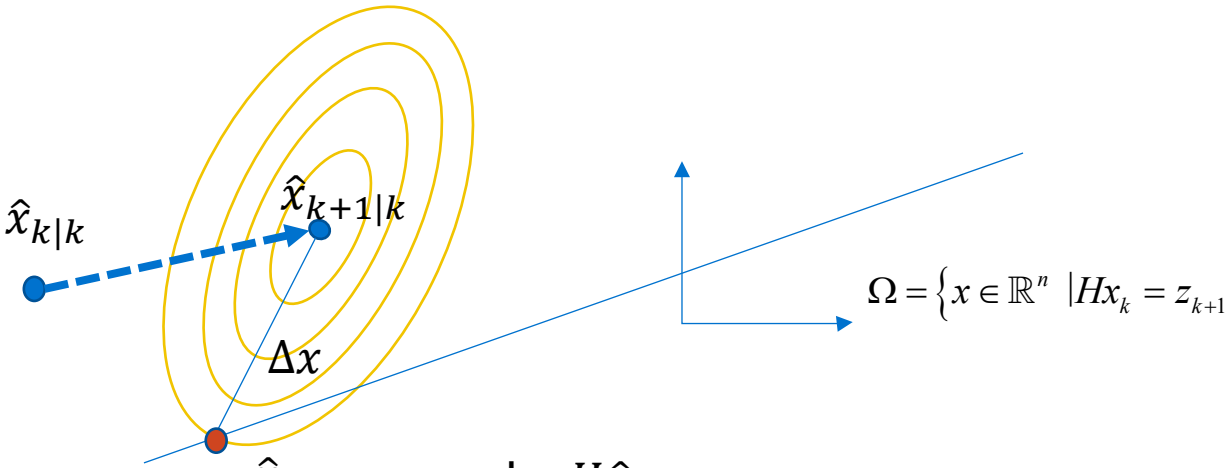
Here we define

$$V_k = \mathrm{E}\left[\mathrm{v}_k\mathrm{v}_k^T\right]$$

Gaussian State Estimate

Now Δx is not strictly perpendicular with Ω but is skewed with by the multivariant estimate $\bar{x}_{k+1}$, Σ_{k+1}.

Need to find the smallest change in x given this new distribution (most probable value of x)



s.t. $H\hat{x}_{k+1|k+1} = z_{k+1}$

Solving for Δx (Multivariate Gaussian Change)

We want to maximize the probability of $p(\boldsymbol{x}) = \frac{1}{(2\pi)^{d/2} \, |\Sigma|^{1/2}} e^{-\frac{1}{2}(\Delta x)^T \Sigma^{-1} (\Delta x)}$

which means minimizing the following value $\| \Delta x \|^2 = \Delta x^T \Sigma^{-1} \Delta x$

And ensure that $H\left(\hat{x}'_{k+1} + \Delta x\right) \in \Omega$ i.e. $H\hat{x}_{k+1} = z_{k+1}$

Which gives us a new orthogonal vector

$$a\Sigma'^{-1}_{k+1} \Delta x = 0$$

Using the same approach as before this gives

$$\Delta x = \Sigma'_{k+1} H \gamma \qquad \text{or} \qquad \Delta x = \Sigma'_{k+1} H^T K v$$

Solving for K

Given

$$\Delta x = \Sigma'_{k+1} H^T K v$$

We can now solve for K as before

$$H\left(\hat{x}'_{k+1|k} + \Sigma'_{k+1} H^T K v\right) = z_{k+1}$$

$$H\Sigma'_{k+1}\left(H^T K v\right) = v$$

Therefore

$$K = \left(H\Sigma_t H^T\right)^{-1}$$

Two-Step Process for Model with Process Error

Prediction

$$\hat{x}'_{k+1} = A\hat{x}_k + Bu_k$$
$$\Sigma'_{k+1} = A\Sigma_k A^T + V_k$$

Update

$$\hat{x}_{k+1} = \hat{x}'_{k+1} + Rv$$
$$\Sigma_{k+1} = \Sigma'_{k+1} - RH\Sigma'_{k+1}$$

Where

$$R = \Sigma'_{k+1} H^T \left(H\Sigma'_{k+1} H^T \right)^{-1}$$

Matrix inverse

Kalman Filter

Seminal paper published in 1960

- Great web page at http://www.cs.unc.edu/~welch/kalman/
- Excellent textbook: Brown and Hwang "Introduction to Random Signals and Applied Kalman Filtering"
- Wikipedia has a helpful discussion: https://en.wikipedia.org/wiki/Kalman_filter

Recursive solution for discrete linear filtering problems

- A state $x \in \mathbb{R}^n$
- A measurement $z \in \mathbb{R}^m$
- Discrete (i.e. for time t_k with k = 1, 2, 3, …)
- Recursive process (i.e. $x_k = f\left(x_{k-1}\right)$)
- Linear system (i.e. $x_k = AX_{k-1}$)

Kalman Filter (2)

Prior estimate $\hat{x}_k^{'}$ at step k: Use the process model to predict what will be the next state of the robot

Posterior estimate $\hat{x}_k$ at step k: Use the observation model to correct the prediction by using the sensor measurement

Computer posterior estimate as a linear combination of the prior estimate and difference between the actual measurement and the expected measurement

$$\widehat{x_k} = \hat{x}_k^{'} + K_k \left(z_k - H\hat{x}_k^{'} \right)$$

Where the Kalman gain K_k is the blending factor that adds a measurement innovation

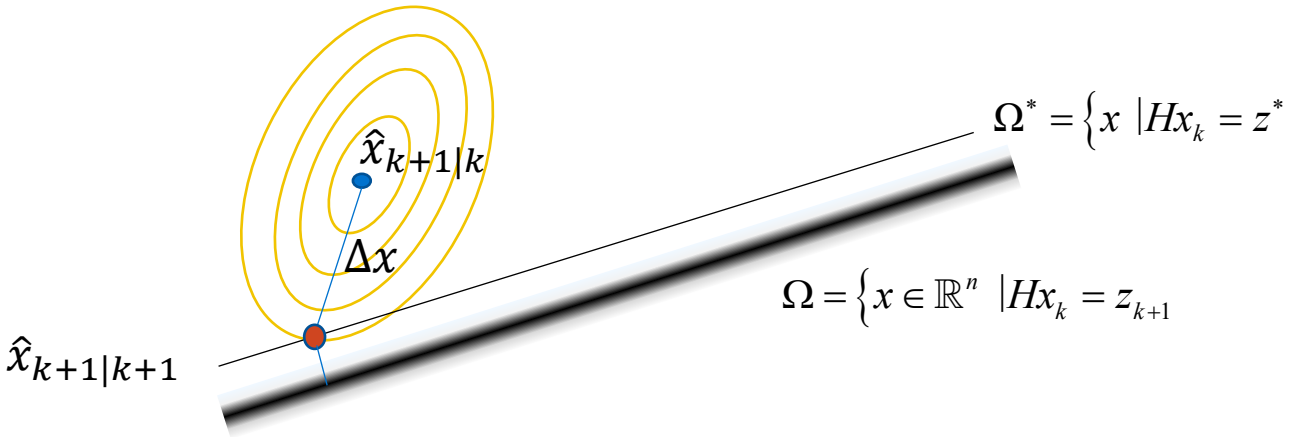
Kalman Filter Observations

Adding noise back into the sensor model

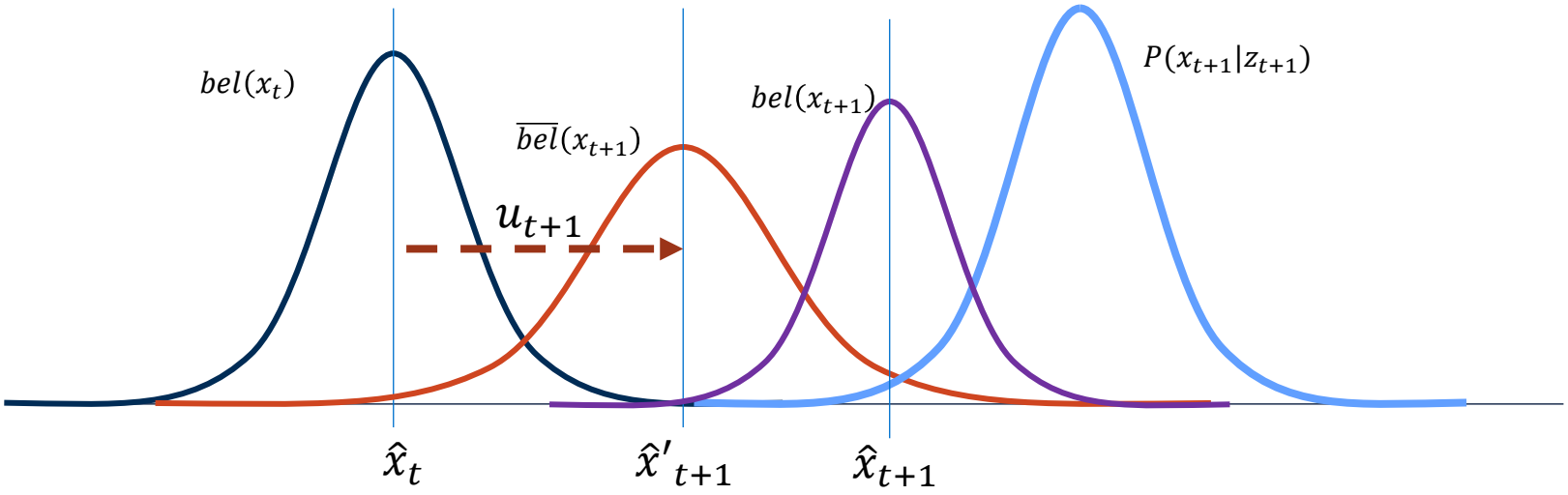
$$x_{k+1} = Ax_k + Bu_k + \mathrm{v}_k$$

$$z_k = Hx_k + \mathrm{w}_k$$

The sensor noise is now projected into the state space.
Must identify the most likely sensor output y^* which is determined by the gaussian distribution on y_{k+1}



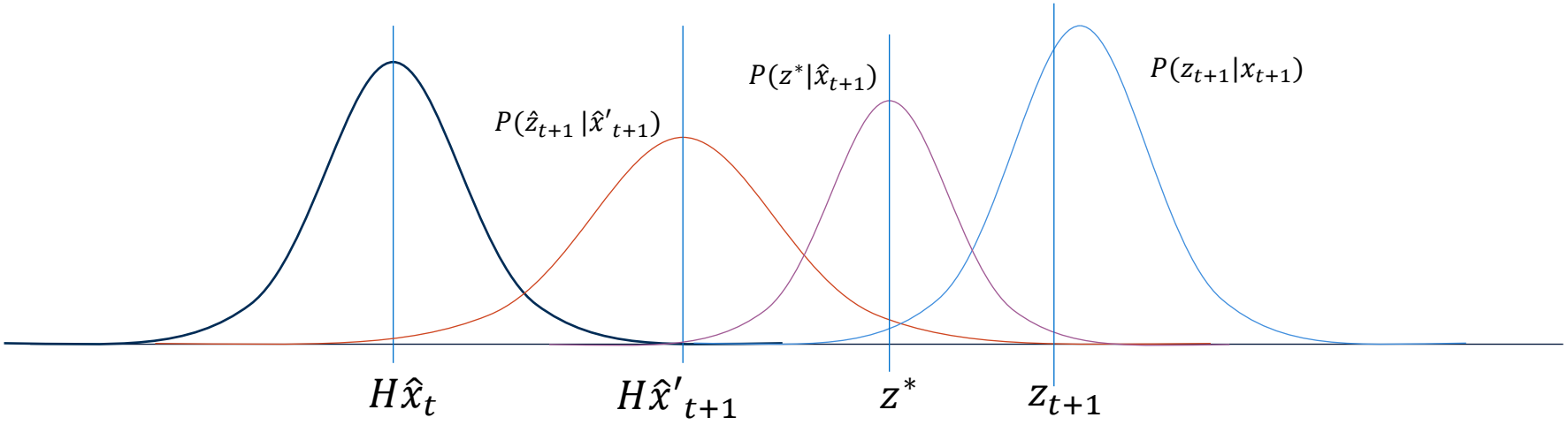
1D Kalman Filter – Bayes Rule



$$bel(x_{t+1}) = \eta P(z_{t+1}|x_{t+1})\overline{bel}(x_{t+1})$$

(Gaussian) (Gaussian) (Gaussian)

1D Kalman Filter – Sensor Space



What if we want to find the most likely output z^*

Which is given by the product $P(z^*|\hat{x}_{t+1}) = P(\hat{z}_{t+1}|\hat{x}'_{t+1})P(z_{t+1}|x_{t+1})$

where $\hat{z}_{t+1} = H\hat{x}'_{t+1}$

Kalman Filter Equations

The predicted sensor value is given by our prior estimate. With the mean value

$$\hat{z} = H\hat{x}'_{k+1}$$

And the covariance matrix

$$\widehat{W} = \mathrm{E}[(\hat{z} - z_{k+1})(\hat{z} - z_{k+1})^T]$$

$$= E\left[H\left(\hat{x}'_{k+1} - x_{k+1}\right)\left(\hat{x}'_{k+1} - x_{k+1}\right)^T H^T \right]$$

$$= H\Sigma'_{k+1}H^T$$

Now we just need to find the new mean of

$$P(z^*|\hat{x}_{t+1}) = P(\hat{z}_{t+1}|\hat{x}'_{t+1})P(z_{t+1}|x_{t+1})$$

Kalman Filter – Gaussian Product

Remembering the rule for Gaussian Products

$$\left.\begin{array}{l} X_1 \sim N\left(\mu_1, \sigma_1^2\right) \\ X_2 \sim N\left(\mu_2, \sigma_2^2\right) \end{array}\right\} \rightarrow X_1 X_2 \sim N\left(\mu_1 + Q\left(\mu_2 - \mu_1\right), \sigma_1 - Q\sigma_1\right)$$

$$Q = \sigma_1\left(\sigma_1 + \sigma_2\right)^{-1}$$

We can represent the probability distributions as

$$P(\hat{z}_{t+1}|\hat{x}'_{t+1}) \sim N(\hat{z}_{t+1}, \widehat{W})$$

$$P(z_{t+1}|x_{t+1}) \sim N(z_{t+1}, W)$$

Giving us a new mean and covariance

$$z^* = \hat{z} + \widehat{W}\left(\widehat{W} + W\right)^{-1}(\hat{z} - z_{k+1})$$

$$P(z^*|\hat{x}_{t+1}) \sim N(z^*, W^*)$$

$$W^* = \widehat{W} + \widehat{W}\left(\widehat{W} + W\right)^{-1} W$$

Kalman Filter Final Steps

Now that we have the value of $z^* = \hat{z} + \widehat{W}\left(\widehat{W} + W_{k+1}\right)^{-1}(\hat{z} - z_{k+1})$

We can define $\Omega^* = \{x \in \mathbb{R}^n | Hx = z^*\}$

We once again seek to minimize

$$\| \Delta x \|^2 = \Delta x^T \Sigma^{-1} \Delta x \qquad \text{Where} \qquad \Delta x = \hat{x}_{\mathrm{k+1}} - \hat{x}'_{\mathrm{k+1}}$$

Which gives us

$$\Delta x = \Sigma'_{k+1} H^T (H \Sigma'_{k+1} H^T)^{-1} (z^* - H \hat{x}'_{k+1})$$

Innovation Error

$$\Delta x = \Sigma'_{k+1} H^T (H \Sigma'_{k+1} H^T + W_{k+1})^{-1} v$$

$$v = z_{k+1} - H \hat{x}'_{k+1}$$

Therefore

$$\hat{x}_{\mathrm{k+1}} = \hat{x}'_{\mathrm{k+1}} + \Sigma'_{k+1} H^T (H \Sigma'_{k+1} H^T + W_{k+1})^{-1} v$$

Kalman Filter (3)

Given the process and measurement models

$$x_{k+1} = Ax_k + Bu_k + v_k$$

$$z_k = Hx_k + w_k$$

$$\mathrm{v_k} \sim N(0,V), \mathrm{w_k} \sim N(0,W)$$

- Start with an initial Guess

$$\hat{x}_0, \Sigma_0$$

- Compute prior (prediction)

$$\widehat{x'_k} = A\hat{x}_{k-1} + Bu_{k-1}$$

$$\Sigma'_k = A\Sigma_{k-1}A^T + V_k$$

- Compute posterior (correction)

$$\hat{x}_k = \hat{x}'_k + \mathrm{R}\Delta z$$

$$\Sigma_k = \Sigma'_\mathrm{k} - RH\Sigma'_\mathrm{k}$$

$$\Delta z = z_{k+1} - Hx'_k$$

$$S = H\Sigma'_k H^T + W_k$$

$$R = \Sigma'_k H^T S^{-1}$$

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.

JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Robotics

Extended Kalman Filter

Kalman Filter Limitations

Assumptions:

- Linear process model
- Linear Observation model
- White Gaussian Noise

What can we do if the system is not linear?

- Non-linear state dynamics. $\tilde{x}_{k+1} = f\left(\hat{x}_k, u_k, w_k\right)$
- Non-linear observations. $\tilde{z}_k = h\left(\tilde{x}_k, v_t\right)$

Linearize It

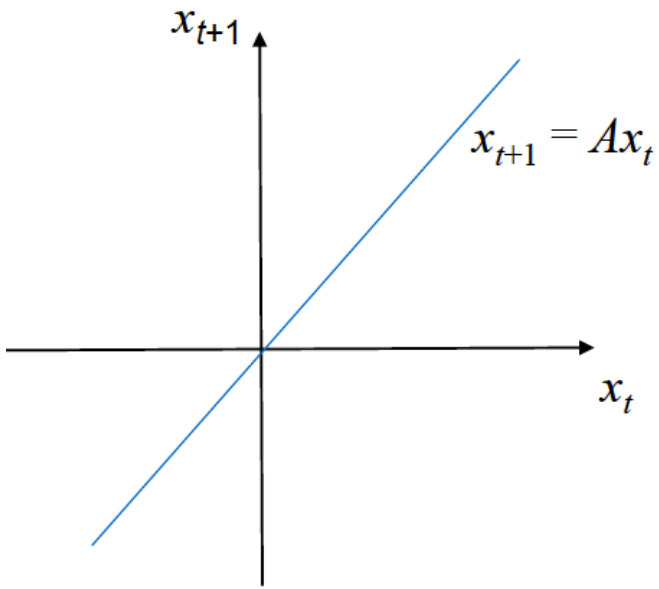
$$x_{k+1} \approx \tilde{x}_{k+1} + A\left(x_k - \hat{x}_k\right) + V_k$$

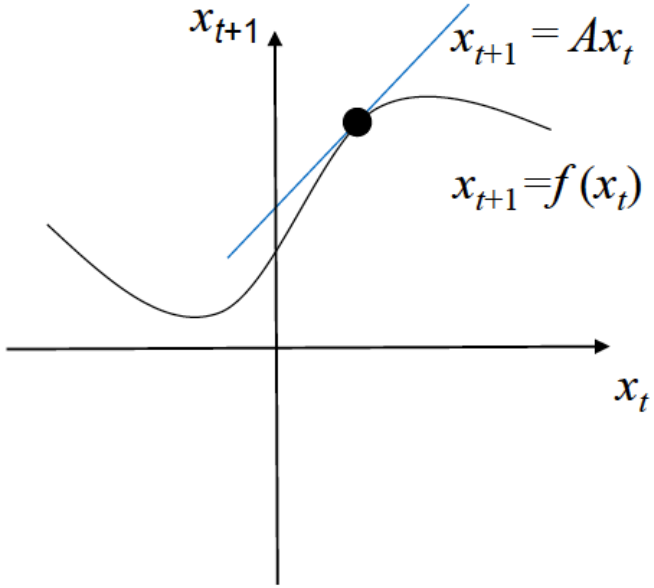
$$\tilde{x}_{k+1} = f\left(\hat{x}_k, u_k, 0\right)$$

$$z_k \approx \tilde{z}_k + H\left(x_k - \widetilde{x_k}\right) + W_k$$

$$\tilde{z}_k = h\left(\tilde{x}_k, 0\right)$$

KF vs EKF





The Jacobian

- A matrix of all the first-order partial derivatives. i.e. The first order Taylor Expansion

$$J = \frac{d\mathbf{f}}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial}{\partial x_1}\mathbf{f} & \cdots & \frac{\partial}{\partial x_1}\mathbf{f} \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1 & \frac{\partial}{\partial x_2} f_1 & & \frac{\partial}{\partial x_N} f_1 \\ \frac{\partial}{\partial x_1} f_2 & \frac{\partial}{\partial x_2} f_2 & \cdots & \frac{\partial}{\partial x_N} f_2 \\ \frac{\partial}{\partial x_1} f_N & \frac{\partial}{\partial x_2} f_N & \cdots & \frac{\partial}{x_N} f_N \end{bmatrix}$$

Linearizing the Process

We can compute the A and B are Jacobians as follows

$$f(x,u,v)=\begin{bmatrix} f_1(x,y,v) \\ \dots. \\ f_m(x,u,v) \end{bmatrix}$$

$$A(x_t)=\begin{bmatrix} \frac{\partial f_1(x_t,u_t,0)}{\partial x_1} & \cdots & \frac{\partial f_1(x_t,u_t,0)}{\partial x_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_M(x_t,u_t,0)}{\partial x_1} & \cdots & \frac{\partial f_M(x_t,u_t,0)}{\partial x_M} \end{bmatrix} \qquad B(x_t)=\begin{bmatrix} \frac{\partial f_1(x_t,u_t,0)}{\partial u_1} & \cdots & \frac{\partial f_1(x_t,u_t,0)}{\partial u_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_M(x_t,u_t,0)}{\partial u_1} & \cdots & \frac{\partial f_M(x_t,u_t,0)}{\partial u_M} \end{bmatrix}$$

Linearizing the Measurement

We can also compute the H matrix in the same fashion

$$h(x,w) = \begin{bmatrix} h_1(x,w) \\ \dots. \\ h_N(x,w) \end{bmatrix} \qquad H\left(x_t\right) = \begin{matrix} \dfrac{\partial h_1\left(x_t,0\right)}{\partial x_1} & \cdots & \dfrac{\partial h_1\left(x_t,0\right)}{\partial x_M} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial h_N\left(x_t,0\right)}{\partial x_1} & \cdots & \dfrac{\partial h_N\left(x_t,0\right)}{\partial x_M} \end{matrix}$$

Extended Kalman Filter

Kalman Filter

Given

$$\hat{x}_0, \Sigma_0$$

Prediction $\hat{x}_1^{'}$ from $\hat{x}_0$ and u_0

$$\begin{aligned} \hat{x}_k^{'} &= A\hat{x}_{k-1} + Bu_{k-1} \\ \Sigma_k^{'} &= A\Sigma_{k-1}A^T + V_k \end{aligned}$$

Correction using z_1

$$\hat{x}_k = \hat{x}_k^{'} + R\Delta y$$

$$\Sigma_t = \Sigma_{\mathrm{k}}^{'} - RH\Sigma_{\mathrm{k}}^{'}$$

$$\Delta y = y_{k+1} - Hx_k'$$

$$S = H_k\Sigma_k' H_k^T + W_k$$

$$R = \Sigma_k' H_k^T S^{-1}$$

Extended Kalman Filter

Given

$$\hat{x}_0, \Sigma_0$$

Prediction $\hat{x}_1^{'}$ from $\hat{x}_0$ and u_0

$$\begin{aligned} \widehat{x_k}^{'} &= f\left(\hat{x}_{k-1}, u_{k-1}, 0\right) \\ \Sigma_k^{'} &= A_k\Sigma_{k-1}A_k^T + V_k \end{aligned}$$

Correction using z_1

$$\hat{x}_k = \hat{x}_k^{'} + R\Delta y$$

$$\Sigma_t = \Sigma_{\mathrm{k}}^{'} - RH\Sigma_{\mathrm{k}}^{'}$$

$$\Delta y = y_{k+1} - h\left(x_k^{'}, 0\right)$$

$$S = H_k\Sigma_k' H_k^T + W_k$$

$$R = \Sigma_k' H_k^T S^{-1}$$

Extended Kalman Filter (2)

A few important points

- You use the non-linear process function to predict the next step
- This prediction uses zero noise
- The predicted measurement must be made with the actual measurement function
- The predicted measurement uses zero noise

Extended Kalman Filter

Given

$$\hat{x}_0, \Sigma_0$$

Prediction $\hat{x}_1^{'}$ from $\hat{x}_0$ and u_0

$$\widehat{x_k}^{'} = f\left(\hat{x}_{k-1}, u_{k-1}, 0\right)$$
$$\Sigma_k^{'} = A_k \Sigma_{k-1} A_k^T + V_k$$

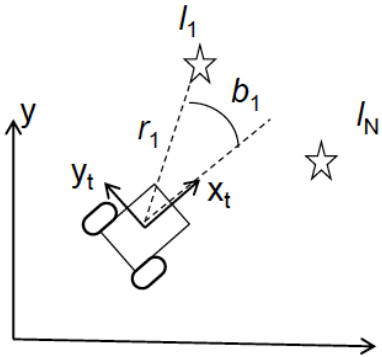
Correction using z_1

$$\hat{x}_k = \hat{x}_k^{'} + R\Delta y$$
$$\Sigma_t = \Sigma_{\mathrm{k}}^{'} - RH\Sigma_{\mathrm{k}}^{'}$$
$$\Delta y = y_{k+1} - h\left(x_k^{'}, 0\right)$$
$$S = H_k \Sigma_k^{'} H_k^T + W_k$$
$$R = \Sigma_k^{'} H_k^T S^{-1}$$

EKF for Range-Bearing Localization

- State $s = \left[x_t, y_t, \theta_t\right]^T$ 2D position and orientation
- Input $u_t = \left[c_t, \omega_t\right]^T$ linear and angular velocity
- Process Model

$$f\left(s_k, u_k, v_k\right) = \begin{bmatrix} x_{k-1} + (\Delta t) c_{k-1} \cos\left(\theta_{k-1}\right) \\ y_k + (\Delta t) c_{k-1} \sin\left(\theta_{k-1}\right) \\ \theta_{k-1} + (\Delta t)\omega_{k-1} \end{bmatrix} + \begin{bmatrix} v_{x_k} \\ v_{y_k} \\ v_{\theta_k} \end{bmatrix}$$



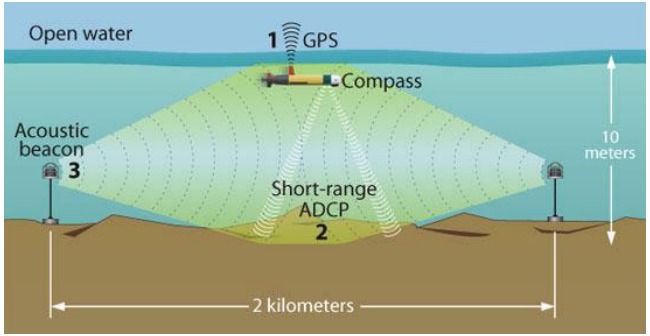
- Given a map the robot sees N landmarks with coordinates

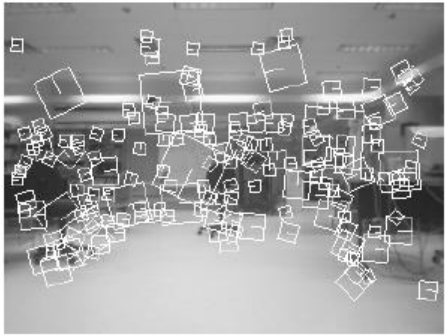
$$l_1 = \left[x_{l1}, y_{l1}\right]^T, \quad l_N = \left[x_{lN}, y_{lN}\right]^T$$

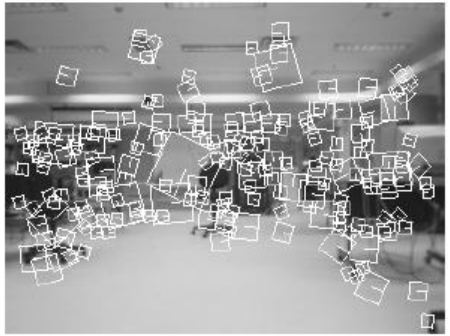
- The observation model is

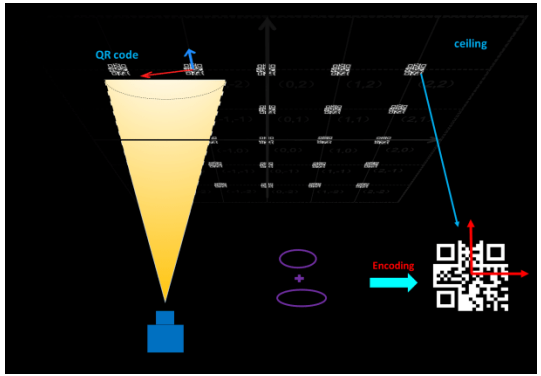
$$z_k = \begin{bmatrix} h_1\left(s_k, w_{1,k}\right) \\ \cdots \\ h_N\left(s_k, w_{N,k}\right) \end{bmatrix} h_i\left(s_t, v_t\right) = \begin{bmatrix} \sqrt{\left(x_k - x_{li}\right)^2 + \left(y_k - y_{li}\right)^2} \\ atan2\left(y_k - y_{li}, x_k - x_{li}\right) - \theta_k \end{bmatrix} + \begin{bmatrix} w_{r_k} \\ w_{b_k} \end{bmatrix}$$

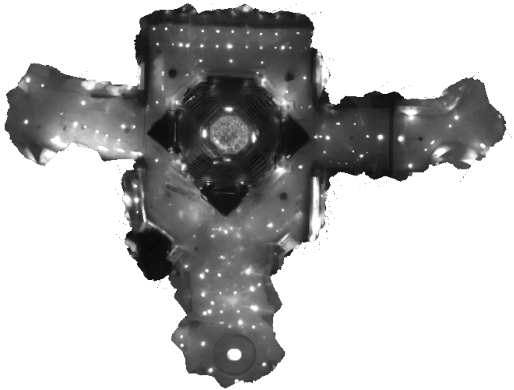
Beacon Localization











EKF –Example Linearize Process Model

$$f\left(s_k, u_k, v_k\right) = \begin{bmatrix} x_{k-1} + (\Delta t) c_{k-1} \cos\left(\theta_{k-1}\right) \\ y_k + (\Delta t) c_{k-1} \sin\left(\theta_{k-1}\right) \\ \theta_{k-1} + (\Delta t)\omega_{k-1} \end{bmatrix} + \begin{bmatrix} v_{x_k} \\ v_{y_k} \\ v_{\theta_k} \end{bmatrix}$$

$$A\left(x_k\right) = \begin{bmatrix} \frac{\partial f_1\left(\widehat{x_k}, u_k, 0\right)}{\partial x_1} & \cdots & \frac{\partial f_1\left(\widehat{x_k}, u_k, 0\right)}{\partial x_M} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_M\left(\widehat{x_k}, u_k, 0\right)}{\partial x_1} & \cdots & \frac{\partial f_M\left(\widehat{x_k}, u_k, 0\right)}{\partial x_M} \end{bmatrix}$$

Exercises:

- Compute the B matrix
  - If the input is $[v, \omega]$ forward speed and turn rate
  - If the input is $[\phi_{left}, \phi_{right}]$ the left and right wheel speeds

$$A\left(x_t\right) = \begin{bmatrix} 1 & 0 & -\Delta t c_{k-1} \sin\theta_{k-1} \\ 0 & 1 & \Delta t c_{k-1} \cos\theta_{k-1} \\ 0 & 0 & 1 \end{bmatrix}$$

Linear Observation Model

$$z = \begin{bmatrix} h_1(s_k, v_N) \\ \dots \\ h_N(s_k, v_N) \end{bmatrix}, \quad h_i(s_k, v_k) = \begin{bmatrix} \sqrt{(x_k - x_{li})^2 + (y_k - y_{li})^2} \\ \tan^{-1}\left(\dfrac{y_k - y_{li}}{x_k - x_{li}}\right) - \theta_k \end{bmatrix} + \begin{bmatrix} \mathrm{w}_{r_k} \\ \mathrm{w}_{b_k} \end{bmatrix}$$

$$H(s_k) = \begin{matrix} \dfrac{\partial h_1(s_k,0)}{\partial x_k} & \dfrac{\partial h_1(s_k,0)}{\partial y_k} & \dfrac{\partial h_1(s_k,0)}{\partial \theta_k} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial h_N(s_k,0)}{\partial x_k} & \dfrac{\partial h_N(s_k,0)}{\partial y_k} & \dfrac{\partial h_N(s_k,0)}{\partial \theta_k} \end{matrix}$$

Some helpful derivatives

$$\frac{\partial}{\partial x}\left(\sqrt{(x-a)^2 + (y-b)^2}\right) = \frac{x-a}{\sqrt{(x-a)^2 + (y-b)^2}}$$

$$\frac{\partial}{\partial x}\left(\tan^{-1}\left(\frac{y-a}{x-b}\right) + c\right) = \frac{a-y}{A^2 - 2ay + b^2 - 2bx + x^2 + y^2}$$

$$\frac{\partial}{\partial y}\left(\sqrt{(x-a)^2 + (y-b)^2}\right) = \frac{y-b}{\sqrt{(x-a)^2 + (y-b)^2}}$$

$$\frac{\partial}{\partial y}\left(\tan^{-1}\left(\frac{y-a}{x-b}\right) + c\right) = \frac{x-b}{a^2 - 2ay + b^2 - 2bx + x^2 + y^2}$$

Unscented Kalman Filter

Motivation- where Extended Kalman Filters Fail

Problems with EKF

-Linearizing a non-linear function can lead to errors

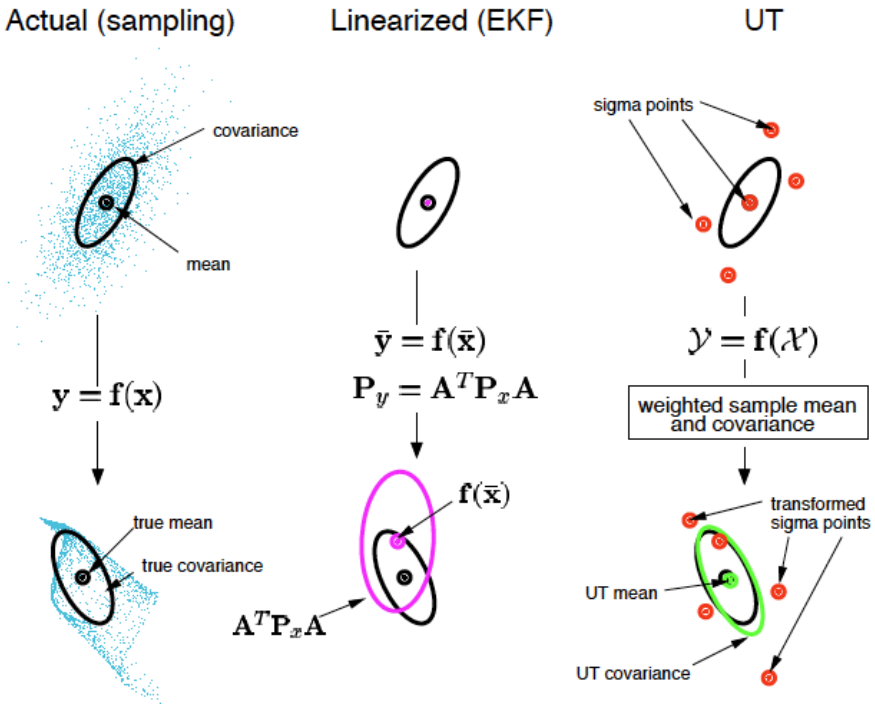
-What if a derivative is not readily available?

Unscented Kalman filter

Estimate the mean and covariance by carefully selected sigma points

EKF approximates f to first order and ignores higher-order terms

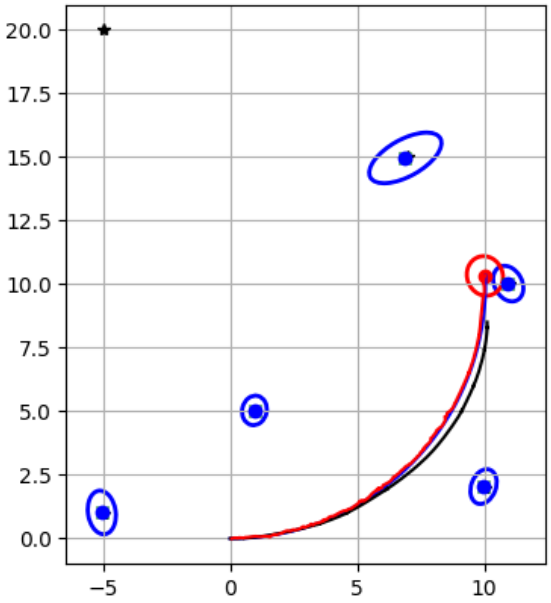
UKF uses f exactly, but approximates p(x).



Wan, Eric A., and Rudolph Van Der Merwe. "The unscented Kalman filter for nonlinear estimation." In *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No. 00EX373)*, pp. 153-158. Ieee, 2000.

Kalman Filters and SLAM

- Localization: state is the location of the robot
- Mapping: state is the location of 2D landmarks
- SLAM: state combines both
- If the state is $s_k = [x_k \; y_k \; \theta_k \; l_{1_k}^T \; l_{2_k}^T \; \dots \; l_{N_k}^T]$ then we can write a linear observation system



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.